

Circle Drawing Algorithms

A Complete University-Level Study Guide

Computer Graphics Course

Understanding How Computers Draw Circles

Learning Objectives

After completing this guide, you will be able to:

1. Explain the mathematical foundations of circle representation
2. Apply 8-way symmetry to optimize circle drawing
3. Implement four different circle drawing algorithms
4. Analyze the trade-offs between algorithms (speed vs. accuracy)
5. Understand why Bresenham's algorithm is preferred in practice

Table of Contents

Section 1: Mathematical Foundations

Circle Equations and Why They Matter

Section 2: Circle Symmetry

The Magic of 8-Way Symmetry

Section 3: Direct Cartesian Method

The Naive Approach

Section 4: Polar Methods

Using Angles to Draw Circles

Section 5: Bresenham's Algorithm

The Gold Standard

Section 6: Comparative Analysis

When to Use Which Algorithm

Section 7: Key Takeaways

Summary and Final Quiz

Section 1: Mathematical Foundations

Circle Equations and Why They Matter

Before we can draw a circle on a computer screen, we need to understand what a circle actually is from a mathematical perspective. A circle is defined as the set of all points that are at a fixed distance (the radius) from a fixed point (the center). This seemingly simple definition gives rise to two powerful mathematical representations that form the foundation of all circle drawing algorithms.

1.1 The Cartesian Equation

The Cartesian equation of a circle is derived directly from the distance formula. If the center is at point (x_c, y_c) and the radius is R , then every point (x, y) on the circle satisfies the equation: $(x - x_c)^2 + (y - y_c)^2 = R^2$. This equation tells us that the sum of the squared distances from any point on the circle to the center, measured along the x and y axes, always equals the square of the radius.

Solving for y, we get: $y = y_c + \sqrt{R^2 - (x - x_c)^2}$. This formula allows us to calculate the y-coordinate for any given x-coordinate. However, notice that we need to compute a square root for every point, which is computationally expensive. The square root operation is one of the most time-consuming arithmetic operations a computer can perform, making this approach potentially slow for real-time graphics applications.

Intuition - Why This Matters:

Think of the Cartesian equation as a "translator" that converts x-values into y-values. It is like having a rule that says: "If you tell me how far right you want to go (x), I will tell you how far up you need to be (y) to stay on the circle." The square root is the price we pay for this translation - it is the mathematical operation that converts the squared distance back to actual distance. In computer graphics, avoiding square roots is often the key to faster algorithms.

1.2 The Parametric (Polar) Equation

The parametric equation takes a completely different approach. Instead of relating x and y directly, it introduces a third variable theta, which represents an angle. The equations are: $x = x_c + R * \cos(\theta)$ and $y = y_c + R * \sin(\theta)$. As theta varies from 0 to 2π radians (or 0 to 360 degrees), the point (x, y) traces out the entire circle.

The beauty of the parametric form is that x and y are expressed independently in terms of theta. This means we can compute both coordinates simultaneously and uniformly sample points around the circle. However, computing sine and cosine functions is also expensive, as these are typically calculated using infinite series expansions: $\sin(\theta) = \theta - \theta^3/3! + \theta^5/5! - \dots$ and $\cos(\theta) = 1 - \theta^2/2! + \theta^4/4! - \dots$

Real-World Example:

Imagine you are at the center of a circular running track. The Cartesian approach is like placing markers every meter along a straight line from the center, then calculating how far along the perpendicular direction you need to go to reach the track. The parametric approach is like standing at the center, pointing your arm at different angles, and walking the radius distance in each direction. Both get you to points on the track, but the angle-based approach feels more natural and uniform.

1.3 Choosing the Right Octant

An octant is one-eighth of a circle. When drawing circles, we do not compute all points directly. Instead, we compute points in one octant and use symmetry to generate the rest. But which octant should we choose? The key insight comes from analyzing the slope of the circle. Taking the derivative of the circle equation with respect to x gives us the slope: $dy/dx = -x/y$. For the slope to have magnitude less than or equal to 1, we need $|x/y|$ less than or equal to 1, which means $|x|$ is less than or equal to $|y|$. Therefore, we compute points in the octant where x is less than or equal to y , starting from $(0, R)$ and continuing until $x = y$.

Mini Quiz 1: Mathematical Foundations

- Q1. Why is the square root operation considered expensive in computer graphics?
- Q2. What is the range of theta needed to trace a complete circle in parametric form?
- Q3. If a circle has center (3, 4) and radius 5, what is the Cartesian equation?
- Q4. Why do we choose the octant where x is less than or equal to y for computation?

Answers:

- A1. Square root requires iterative approximation algorithms and cannot be computed in a fixed number of operations like addition or multiplication.
- A2. Theta must vary from 0 to 2π radians (approximately 6.28 radians) to trace the complete circle.
- A3. $(x-3)^2 + (y-4)^2 = 25$
- A4. In this octant, the slope magnitude is at most 1, ensuring each x step corresponds to at most one y step, preventing gaps in the drawn circle.

Section 2: Circle Symmetry

The Magic of 8-Way Symmetry

One of the most elegant optimizations in computer graphics is the exploitation of circle symmetry. A circle centered at the origin exhibits 8-way symmetry, meaning that if you know one point (a, b) on the circle, you automatically know seven other points. This property reduces the computational burden by a factor of eight, making circle drawing algorithms significantly faster.

2.1 Understanding the Eight Symmetric Points

If point (a, b) lies on a circle centered at the origin with radius R , then $a^2 + b^2 = R^2$. Due to the properties of squaring (both positive and negative values give the same square), we can manipulate the signs and positions of a and b in eight different ways while still satisfying the circle equation. The eight symmetric points are: (a, b) , $(-a, b)$, $(-a, -b)$, $(a, -b)$, (b, a) , $(-b, a)$, $(-b, -a)$, and $(b, -a)$.

Let us understand why this works. The first four points come from sign changes: if (a, b) is on the circle, then so are $(-a, b)$ [reflected across y-axis], $(-a, -b)$ [reflected across origin], and $(a, -b)$ [reflected across x-axis]. The next four points come from swapping x and y coordinates: (b, a) [reflected across line $y=x$] and its three sign variations. All eight points maintain the same distance R from the origin because $a^2 + b^2 = (-a)^2 + b^2 = b^2 + a^2 = R^2$.

Symmetry Relationships:

Original Point	Transformation	Resulting Point
(a, b)	No change (original)	(a, b)
(a, b)	Reflect across y-axis	$(-a, b)$
(a, b)	Reflect across origin	$(-a, -b)$
(a, b)	Reflect across x-axis	$(a, -b)$
(a, b)	Swap x and y	(b, a)
(a, b)	Swap and reflect y-axis	$(-b, a)$
(a, b)	Swap and reflect origin	$(-b, -a)$
(a, b)	Swap and reflect x-axis	$(b, -a)$

2.2 Handling General Circle Centers

When the circle center is at a general point (x_c, y_c) rather than the origin, we simply add the center coordinates to each symmetric point. So the eight points become: $(x_c + a, y_c + b)$, $(x_c - a, y_c + b)$, $(x_c - a, y_c - b)$, $(x_c + a, y_c - b)$, $(x_c + b, y_c + a)$, $(x_c - b, y_c + a)$, $(x_c - b, y_c - a)$, and $(x_c + b, y_c - a)$. This is equivalent to first computing points on a circle centered at the origin, then translating (shifting) all points by the center coordinates.

Intuition - Why This Saves Work:

Imagine you need to draw a circular fence around a garden. Without symmetry, you would need to walk around the entire circle, measuring and placing posts. With 8-way symmetry, you only need to carefully measure and place posts in one-eighth of the circle (45 degrees), then use a mirror technique to instantly know where the other posts go. You have reduced your work by 87.5%! This is exactly what computers do - calculate points carefully in one octant, then "mirror" them to fill the rest.

Real-World Example:

In a video game rendering a circular radar display, the graphics processor computes only the points in one 45-degree wedge of the radar circle, then uses 8-way symmetry to complete the circle. This optimization allows the game to render the radar at higher frame rates. Similarly, when you see circular progress indicators on websites or mobile apps, they often use this symmetry optimization to ensure smooth animation without consuming excessive processing power.

2.3 The Draw8Points Function

The Draw8Points function is the practical implementation of 8-way symmetry. It takes a single computed point (a, b) relative to the origin and draws all eight symmetric points by applying sign changes and coordinate swaps, then shifting by the center (x_c, y_c).

```
void Draw8Points(HDC hdc, int xc, int yc, int a, int b, COLORREF color)
{
    // Points from (a,b): sign changes on both coordinates
    SetPixel(hdc, xc+a, yc+b, color); // Quadrant I
    SetPixel(hdc, xc-a, yc+b, color); // Quadrant II
    SetPixel(hdc, xc-a, yc-b, color); // Quadrant III
    SetPixel(hdc, xc+a, yc-b, color); // Quadrant IV
    // Points from swapped (b,a): sign changes on swapped coordinates
    SetPixel(hdc, xc+b, yc+a, color); // Octant between I and II
    SetPixel(hdc, xc-b, yc+a, color); // Octant between II and III
    SetPixel(hdc, xc-b, yc-a, color); // Octant between III and IV
    SetPixel(hdc, xc+b, yc-a, color); // Octant between IV and I
}
```

Mini Quiz 2: Circle Symmetry

- Q1. If you compute a point (3, 4) on a circle centered at the origin, list all eight symmetric points.
- Q2. For a circle centered at (10, 20), if one computed point is (2, 3), what are the eight pixels to draw?
- Q3. Why does swapping coordinates (x, y) to (y, x) still give a point on the same circle?
- Q4. How much computational work does 8-way symmetry save?

Answers:

- A1. (3,4), (-3,4), (-3,-4), (3,-4), (4,3), (-4,3), (-4,-3), (4,-3)
- A2. (12,23), (8,23), (8,17), (12,17), (13,22), (7,22), (7,18), (13,18)
- A3. Because $x^2 + y^2 = y^2 + x^2$ (addition is commutative), so the distance from origin remains R.
- A4. 87.5% - you compute 1/8 of the points and derive the rest through symmetry.

Section 3: Direct Cartesian Method

The Naive Approach - Understanding the Basics

The Direct Cartesian method is the most straightforward approach to drawing circles - it directly applies the Cartesian equation we learned earlier. While not the most efficient method, understanding it is crucial because it reveals the fundamental challenges that more sophisticated algorithms address. This method is often called the "naive" approach because it solves the problem directly without any clever optimizations.

3.1 Algorithm Overview

The algorithm works as follows: Starting from $x = 0$ (the top of the circle), we increment x step by step. For each x value, we calculate y using the equation $y = \sqrt{R^2 - x^2}$. We continue until x equals y (the diagonal line where our chosen octant ends). Each computed point is passed to `Draw8Points` to exploit symmetry.

The choice of stopping when $x = y$ is deliberate. This is the boundary of the octant where the slope magnitude equals 1. Beyond this point, the slope would exceed 1 in magnitude, meaning that incrementing x by 1 could cause y to change by more than 1, potentially leaving gaps in the drawn circle. By stopping at $x = y$, we ensure smooth, gap-free drawing within our octant.

3.2 Implementation Details

The implementation is remarkably simple. We precompute R^2 to avoid recalculating it in every iteration. For each x from 0 to the point where x equals y , we compute $y = \sqrt{R^2 - x^2}$ and round to the nearest integer. The rounding is necessary because pixel coordinates must be integers, and it introduces a small amount of error that accumulates around the circle.

```
void CircleDirect(HDC hdc, int xc, int yc, int R, COLORREF color)
{
    int x = 0, y = R;
    int R2 = R * R; // Precompute R-squared
    Draw8Points(hdc, xc, yc, x, y, color);
    while(x < y)
    {
        x++;
        y = round(sqrt((double)(R2 - x*x)));
        Draw8Points(hdc, xc, yc, x, y, color);
    }
}
```

Intuition - Why This Works:

Think of this algorithm as "walking" along the top of the circle from left to right. At each step, you move one unit to the right (increment x), then look up how far down you need to go (calculate y using the circle equation). The square root tells you the exact distance from the horizontal line $y = R$ to the circle at position x . The result is rounded to find the nearest pixel. While this sounds perfectly logical, the square root operation is like asking a very complex question every time you take a step - it works, but it is slow.

3.3 Performance Analysis

The Direct Cartesian method has several performance characteristics that make it suboptimal for real-time graphics. First, the square root operation is computationally expensive, typically requiring many CPU cycles or even hardware support. Second, floating-point operations (the square root returns a double) are generally slower than integer operations on most processors. Third, the rounding operation introduces a small but cumulative error.

Characteristic	Direct Cartesian	Impact
Operations per pixel	1 sqrt, 1 multiply, 1 subtract	Slow due to sqrt
Integer operations	No (uses floating-point)	Additional overhead
Memory accesses	Minimal	Good cache performance
Accuracy	Good (rounding errors only)	Acceptable for most uses

Real-World Example:

Consider drawing a circle with radius 100 pixels. The algorithm will iterate approximately 71 times (from $x=0$ to x approximately 71 where $x=y$). Each iteration requires one square root calculation. On a typical CPU, a square root might take 10-100 CPU cycles, compared to just 1 cycle for integer addition. For real-time graphics running at 60 frames per second, drawing multiple circles per frame, this overhead becomes significant. Game developers avoid this method in performance-critical code.

Mini Quiz 3: Direct Cartesian Method

- Q1. Why does the algorithm stop when x equals y ?
- Q2. What makes the square root operation problematic in this algorithm?
- Q3. For a circle with $R = 50$, approximately how many iterations will the while loop execute?
- Q4. Why do we precompute $R^2 = R * R$?

Answers:

- A1. This is the boundary where the circle slope magnitude equals 1. Beyond this, the slope exceeds 1, potentially causing gaps.
- A2. Square root is computationally expensive (many CPU cycles), requires floating-point operations, and cannot be computed in constant time like basic arithmetic.
- A3. Approximately 35 iterations (x goes from 0 to about 35, where x is approximately y is approximately $R/\sqrt{2}$ is approximately 35.4).
- A4. To avoid recalculating R^2 in every loop iteration, saving one multiplication per pixel.

Section 4: Polar Methods

Using Angles to Draw Circles

Polar methods take a fundamentally different approach to circle drawing. Instead of incrementing x and calculating y , these methods use the parametric (polar) form of the circle equation, varying an angle θ to generate points uniformly around the circle. This approach has distinct advantages and challenges compared to the Cartesian method.

4.1 Direct Polar Method

The direct polar method directly applies the parametric equations: $x = R * \cos(\theta)$ and $y = R * \sin(\theta)$. Starting from $\theta = 0$ (the point $(R, 0)$ on the positive x -axis), we increment θ by a small amount $d\theta$ and recalculate x and y for each step. The question is: how do we choose $d\theta$?

If $d\theta$ is too large, we will get disconnected points (gaps in the circle). If $d\theta$ is too small, we will compute many points that round to the same pixel, wasting computational resources. The optimal choice comes from understanding arc length. An arc of angle $d\theta$ on a circle of radius R has length $R * d\theta$ (when $d\theta$ is in radians). For an arc length of approximately 1 pixel, we need $d\theta = 1/R$.

```
void CirclePolar(HDC hdc, int xc, int yc, int R, COLORREF color)
{
    int x = R, y = 0;
    double theta = 0, dtheta = 1.0 / R; // Step size for ~1 pixel arcs
    Draw8Points(hdc, xc, yc, x, y, color);
    while(x > y) // Continue until we reach the diagonal
    {
        theta += dtheta;
        x = round(R * cos(theta));
        y = round(R * sin(theta));
        Draw8Points(hdc, xc, yc, x, y, color);
    }
}
```

Intuition - Why $d\theta = 1/R$:

Imagine walking around a circular track. If the track has a small radius, you need to turn more sharply to cover each step. If the track has a large radius, you turn more gradually. The relationship is inverse: $d\theta = 1/R$. For a radius of 100 pixels, each step covers an angle of 0.01 radians (about 0.57 degrees). This ensures that each step moves you approximately 1 pixel along the circle's circumference - not too far (causing gaps) and not too close (wasting computation).

4.2 Iterative Polar Method

The direct polar method has a major inefficiency: computing $\sin$ and $\cos$ in every iteration. These trigonometric functions, like square roots, are computed using infinite series and are slow. The iterative polar method eliminates repeated trigonometric calculations using a clever mathematical trick.

The key insight is that we can compute the next point from the current point without explicitly calling $\sin$ and $\cos$. If the current point is $(x, y) = (R * \cos(\theta), R * \sin(\theta))$, the next point after incrementing by $d\theta$ is $(x', y') = (R * \cos(\theta + d\theta), R * \sin(\theta + d\theta))$. Using angle addition formulas: $\cos(\theta + d\theta) = \cos(\theta) * \cos(d\theta) - \sin(\theta) * \sin(d\theta)$ and $\sin(\theta + d\theta) = \sin(\theta) * \cos(d\theta) + \cos(\theta) * \sin(d\theta)$.

$\sin(\theta) \cdot \cos(d\theta) + \cos(\theta) \cdot \sin(d\theta)$. This gives us: $x' = x \cdot \cos(d\theta) - y \cdot \sin(d\theta)$ and $y' = x \cdot \sin(d\theta) + y \cdot \cos(d\theta)$.

Since $d\theta$ is constant (equal to $1/R$), we only need to compute $\cos(d\theta)$ and $\sin(d\theta)$ once at the beginning! Every subsequent point is calculated using only multiplications and additions - no expensive trigonometric function calls.

```
void CircleIterativePolar(HDC hdc, int xc, int yc, int R, COLORREF color)
{
    double x = R, y = 0;
    double dtheta = 1.0 / R;
    double cdtheta = cos(dtheta), sdtheta = sin(dtheta); // Compute once!
    Draw8Points(hdc, xc, yc, R, 0, color);
    while(x > y)
    {
        double x1 = x * cdtheta - y * sdtheta; // New x from rotation
        y = x * sdtheta + y * cdtheta; // New y from rotation
        x = x1;
        Draw8Points(hdc, xc, yc, round(x), round(y), color);
    }
}
```

Intuition - Rotation Interpretation:

The iterative polar method is actually rotating a point around the origin! Think of it as attaching a string of length R to the origin and swinging it in small steps. Each step rotates the current point by a tiny angle $d\theta$. The formulas $x' = x \cdot \cos(d\theta) - y \cdot \sin(d\theta)$ and $y' = x \cdot \sin(d\theta) + y \cdot \cos(d\theta)$ are the standard 2D rotation formulas. Because the rotation angle is constant, we precompute the $\cos$ and $\sin$ values once, then apply the same rotation repeatedly. This is like having a template for each small rotation step.

Real-World Example:

The iterative polar method is commonly used in robotics and CNC (Computer Numerical Control) machining. When a robotic arm needs to move in a circular path, the controller uses similar rotation calculations to determine each position along the arc. The precomputation of $\cos(d\theta)$ and $\sin(d\theta)$ is particularly valuable in embedded systems with limited processing power, such as the microcontrollers in 3D printers and laser cutters.

4.3 Comparison: Direct vs. Iterative Polar

Aspect	Direct Polar	Iterative Polar
Trig functions per iteration	2 (sin, cos)	0 (precomputed)
Floating-point operations	2 trig + 2 multiply	4 multiply + 2 add
Speed	Slow	Much faster
Accuracy	High	Accumulates small error

Q1. Why do we choose $d\theta = 1/R$ in the polar method?

Q2. What is the main performance problem with the direct polar method?

Q3. How does the iterative polar method avoid computing $\sin$ and $\cos$ in each iteration?

Q4. The formulas $x' = x \cdot \cos(d\theta) - y \cdot \sin(d\theta)$ and $y' = x \cdot \sin(d\theta) + y \cdot \cos(d\theta)$ represent what geometric operation?

Answers:

A1. This ensures each step covers approximately 1 pixel of arc length ($\text{arc length} = R \cdot d\theta = R \cdot (1/R) = 1$), avoiding gaps and redundant pixels.

A2. Computing $\sin(\theta)$ and $\cos(\theta)$ in every iteration is slow because trigonometric functions require infinite series calculations.

A3. It uses angle addition formulas to express new coordinates in terms of current coordinates, precomputing $\cos(d\theta)$ and $\sin(d\theta)$ once at the start.

A4. 2D rotation by angle $d\theta$ around the origin. Each step rotates the current point to get the next point on the circle.

Section 5: Bresenham's Algorithm

The Gold Standard for Circle Drawing

Bresenham's circle algorithm, also known as the midpoint circle algorithm, is the most widely used method for drawing circles in computer graphics. It achieves what every graphics programmer wants: integer-only arithmetic, no expensive operations like square roots or trigonometric functions, and optimal visual quality. Understanding this algorithm unlocks the secret behind efficient circle rendering in everything from video games to operating systems.

5.1 The Core Idea: Midpoint Testing

The key insight behind Bresenham's algorithm is elegantly simple. Instead of calculating exactly where the circle passes, we make a binary decision at each step: which of two candidate pixels is closer to the true circle? We do this by testing a "midpoint" - a point halfway between the two candidates - to see if it lies inside or outside the circle.

Consider we have just drawn a pixel at position (x, y) in our octant. The circle is moving downward and to the right. The next pixel must be either at $(x+1, y)$ [move right] or $(x+1, y-1)$ [move right and down]. To decide, we test the midpoint at $(x+1, y-0.5)$. If this midpoint is inside the circle, the circle passes above the midpoint, so we choose $(x+1, y)$. If the midpoint is outside the circle, the circle passes below, so we choose $(x+1, y-1)$.

Intuition - The Fence Analogy:

Imagine you are building a fence along a curved property line (the circle). At each post, you have two choices for the next post location. You cannot see the exact property line, but you can check if the midpoint between your two options is on your side (inside) or the neighbor's side (outside). If the midpoint is on your side, the property line curves away from you - place the next post in the "upper" position. If the midpoint is on the neighbor's side, the property line curves toward you - place the next post in the "lower" position. This simple test at each step guides you along the curve perfectly!

5.2 The Circle Test Function

To test whether a point is inside or outside the circle, we use the circle function: $F(x, y) = x^2 + y^2 - R^2$. For a circle centered at the origin with radius R : if $F(x, y)$ is less than 0, the point (x, y) is inside the circle. If $F(x, y)$ is greater than 0, the point is outside. If $F(x, y)$ equals 0, the point is exactly on the circle (rare for integer pixels).

For our midpoint test at $(x+1, y-0.5)$, we define a decision variable: $d = F(x+1, y-0.5) = (x+1)^2 + (y-0.5)^2 - R^2$. This expands to: $d = x^2 + 2x + 1 + y^2 - y + 0.25 - R^2$. The 0.25 is a fraction we would rather avoid. Here is the clever part: removing 0.25 does not change the sign of d , and the sign is all we care about! So we use $d = x^2 + 2x + 1 + y^2 - y - R^2$.

5.3 Iterative Decision Variable Update

The true genius of Bresenham's algorithm is that we update d iteratively, never computing squares from scratch. After drawing pixel (x, y) with decision variable d , we need the new decision variable d' for the next midpoint.

Case 1: d less than 0 (midpoint inside circle, choose pixel $(x+1, y)$)

The next midpoint is at $(x+2, y-0.5)$. The new decision variable is:

$$d' = F(x+2, y-0.5) = (x+2)^2 + (y-0.5)^2 - R^2$$

Subtracting d from d' : $\Delta d = d' - d = 2x + 3$

So: $d' = d + 2x + 3$

Case 2: d greater than or equal to 0 (midpoint outside or on circle, choose pixel $(x+1, y-1)$)

The next midpoint is at $(x+2, y-1.5)$. The new decision variable is:

$$d' = F(x+2, y-1.5) = (x+2)^2 + (y-1.5)^2 - R^2$$

Subtracting d from d' : $\Delta d = d' - d = 2(x-y) + 5$

So: $d' = d + 2(x-y) + 5$, and we also decrement y .

5.4 Initial Decision Variable

We start at $(0, R)$, the top of the circle. The first midpoint to test is at $(1, R-0.5)$. The initial decision variable is: $d = F(1, R-0.5) = 1 + (R-0.5)^2 - R^2 = 1 + R^2 - R + 0.25 - R^2 = 1.25 - R$. Again, we can remove the 0.25 without affecting the sign decisions. So $d = 1 - R$ is our initial value.

```
void CircleBresenham(HDC hdc, int xc, int yc, int R, COLORREF color)
{
    int x = 0, y = R;
    int d = 1 - R; // Initial decision variable
    Draw8Points(hdc, xc, yc, x, y, color);
    while(x < y)
    {
        if(d < 0)
            d += 2*x + 2; // Move right: update d
        else
        {
            d += 2*(x - y) + 5; // Move right-down: update d
            y--; // And decrement y
        }
        x++;
        Draw8Points(hdc, xc, yc, x, y, color);
    }
}
```

5.5 The Faster Bresenham Variant

We can make the algorithm even faster by computing the increment values iteratively. Define $c1 = 2x + 3$ (the increment when d is less than 0) and $c2 = 2(x-y) + 5$ (the increment when d is greater than or equal to 0). Instead of recalculating these from scratch, we update them iteratively.

Initially: $c1 = 3$ (when $x = 0$) and $c2 = 5 - 2R$ (when $x = 0, y = R$).

After each step, x increases by 1, so $c1$ increases by 2.

If we moved right-down (d greater than or equal to 0), y also decreases by 1, so $c2$ increases by 4.

If we moved right only (d less than 0), y stays the same, so $c2$ increases by 2.

```
void CircleFasterBresenham(HDC hdc, int xc, int yc, int R, COLORREF color)
{
    int x = 0, y = R;
    int d = 1 - R;
    int c1 = 3, c2 = 5 - 2*R; // Increment values
```

```

Draw8Points(hdc, xc, yc, x, y, color);
while(x < y)
{
    if(d < 0)
    {
        d += c1;
        c2 += 2;
    }
    else
    {
        d += c2;
        c2 += 4;
        y--;
    }
    c1 += 2;
    x++;
    Draw8Points(hdc, xc, yc, x, y, color);
}
}

```

Real-World Example:

Bresenham's algorithm is used in virtually every graphics library and operating system. When you draw a circle in Microsoft Paint, when a video game renders a circular minimap, when a CAD program displays a circular gear - Bresenham's algorithm is likely doing the work behind the scenes. Its combination of integer-only arithmetic and optimal pixel selection makes it ideal for both software rendering and hardware acceleration. Many graphics cards implement Bresenham's algorithm directly in silicon for maximum speed.

Mini Quiz 5: Bresenham's Algorithm

- Q1. What does the midpoint test actually determine?
- Q2. Why can we remove the 0.25 from the decision variable without breaking the algorithm?
- Q3. For $R = 10$, what is the initial value of d ?
- Q4. If $d = -5$ and $x = 3$, $y = 9$, what is the new value of d after one iteration? Which pixel do we draw?
- Q5. Why is Bresenham's algorithm considered superior to the Cartesian and polar methods?

Answers:

- A1. It determines whether the midpoint between two candidate pixels is inside or outside the circle, which tells us which pixel is closer to the true circle.
- A2. The 0.25 only affects the magnitude of d , not its sign. Since we only care whether d is negative or non-negative, removing 0.25 does not change our decisions.
- A3. $d = 1 - R = 1 - 10 = -9$
- A4. d is less than 0, so $d' = d + 2x + 2 = -5 + 6 + 2 = 3$. We draw pixel (4, 9) - moved right, y unchanged.
- A5. It uses only integer arithmetic, avoids expensive operations (sqrt, sin, cos), and produces optimal pixel selection - making it fast and accurate.

Section 6: Comparative Analysis

When to Use Which Algorithm

Now that we understand all four circle drawing algorithms, let us compare them systematically. Each algorithm has distinct characteristics that make it suitable for different scenarios. Understanding these trade-offs is essential for making informed decisions in graphics programming.

6.1 Performance Comparison

Algorithm	Operations/Pixel	Integer?	Speed	Best For
Direct Cartesian	1 sqrt + fp ops	No	Slow	Educational
Direct Polar	2 trig + fp ops	No	Slow	Arc segments
Iterative Polar	4 mul + 2 add	No	Medium	Smooth rotation
Bresenham	2-4 add + compare	Yes	Fast	General use
Faster Bresenham	2-3 add + compare	Yes	Fastest	Real-time graphics

6.2 Choosing the Right Algorithm

Use Direct Cartesian when: You need a simple, intuitive implementation for educational purposes or when performance is not critical. It is also useful when you need to draw partial circles where you can directly specify x-coordinates.

Use Direct Polar when: You need to draw arc segments (partial circles) with specific angular ranges. The polar form naturally allows you to start and stop at any angle, making it ideal for pie charts, progress indicators, and arc drawing tools.

Use Iterative Polar when: You need smooth, uniform point distribution around a circle, such as in animation systems where objects rotate continuously. The iterative approach maintains good performance while providing uniform angular sampling.

Use Bresenham (standard or faster) when: You need the best all-around performance. This is the default choice for most graphics applications, including games, CAD software, operating system graphics, and embedded systems. The integer-only arithmetic makes it particularly suitable for hardware implementation.

Intuition - Speed vs. Flexibility Trade-off:

Think of algorithms as tools in a workshop. Bresenham is like a precision circular saw - fast, accurate, perfect for straight cuts (full circles). Polar methods are like adjustable protractors - flexible for measuring and drawing specific angles, but slower to use. Direct Cartesian is like measuring and marking each point by hand - intuitive but tedious. Choose your tool based on the job: speed for mass production (Bresenham), flexibility for custom work (Polar), simplicity for learning (Cartesian).

Real-World Decision Example:

Consider building a mobile game with a circular radar display. For the static radar circle outline, use Bresenham - it is drawn once per frame and needs to be fast. For the sweeping radar line animation, use iterative polar - it provides uniform angular progression. For the pie-chart-style enemy detection zones, use direct polar - you need to draw arcs from specific angles. Each part of the display uses a different algorithm optimized for its specific requirements.

Section 7: Key Takeaways and Final Assessment

7.1 Core Concepts Summary

1. Mathematical Foundations: Circles can be represented using Cartesian equations (relating x and y) or parametric/polar equations (using angle theta). Each representation leads to different drawing algorithms with distinct performance characteristics.
2. 8-Way Symmetry: Computing one-eighth of a circle and using symmetry to generate the remaining points reduces computation by 87.5%. This optimization is fundamental to all efficient circle drawing algorithms.
3. Algorithm Trade-offs: The Direct Cartesian method is intuitive but slow (uses sqrt). Polar methods are flexible but require trigonometric functions. Bresenham's algorithm achieves optimal performance using only integer arithmetic and simple comparisons.
4. Decision Variable Insight: Bresenham's key innovation is the decision variable d, which tracks whether the algorithm is "inside" or "outside" the ideal circle. Updating d iteratively avoids expensive recalculations.
5. Practical Application: In real-world graphics programming, Bresenham's algorithm is the default choice. Polar methods are reserved for cases requiring specific angular control.

7.2 Algorithm Complexity Summary

Algorithm	Time Complexity	Space Complexity	Integer Ops
Direct Cartesian	O(R) with expensive ops	O(1)	No
Direct Polar	O(R) with expensive ops	O(1)	No
Iterative Polar	O(R) with float ops	O(1)	No
Bresenham	O(R) with cheap ops	O(1)	Yes

7.3 Final Comprehensive Quiz

Final Assessment: Test Your Complete Understanding

- Q1. A circle with center (5, 5) and radius 3 needs to be drawn. Using 8-way symmetry, if one computed point is (1, 3), list all eight pixels that will be drawn.
- Q2. Explain why Bresenham's algorithm uses integer operations only, while the Direct Cartesian method requires floating-point operations.
- Q3. For a circle of radius 50, approximately how many iterations will Bresenham's algorithm perform in

the main while loop?

Q4. You need to draw a circular progress bar that fills from 0 degrees to 270 degrees. Which algorithm would you choose and why?

Q5. Trace Bresenham's algorithm for $R = 5$. Start with $x = 0, y = 5, d = -4$. Determine the first 3 pixels drawn (after the initial point).

Answers to Final Assessment

A1. Adding center (5,5) to symmetric points of (1,3):

$(5+1, 5+3) = (6, 8), (5-1, 5+3) = (4, 8), (5-1, 5-3) = (4, 2), (5+1, 5-3) = (6, 2)$

$(5+3, 5+1) = (8, 6), (5-3, 5+1) = (2, 6), (5-3, 5-1) = (2, 4), (5+3, 5-1) = (8, 4)$

A2. Bresenham's algorithm avoids floating-point by: (1) using a decision variable that only needs sign testing, (2) carefully designing the increments to be integers, and (3) removing fractional constants (0.25) that do not affect sign decisions. The Cartesian method must compute $\sqrt{R^2 - x^2}$, which inherently produces non-integer results requiring floating-point representation.

A3. Approximately $R/\sqrt{2}$ or approximately 35 iterations. The loop continues while x is less than y , which means until the diagonal line $x = y$. On this line, $x = y = R/\sqrt{2}$ which is approximately 35.4 for $R = 50$.

A4. Direct Polar method. It naturally supports angular range specification - you can set θ from 0 to $3\pi/2$ (270 degrees) and draw exactly that arc. Bresenham would require modification to handle partial circles.

A5. Tracing for $R = 5$:

Start: $x=0, y=5, d=-4$

Iteration 1: $d=-4$ less than 0, so $d = -4 + 2(0) + 2 = -2, x=1, y=5$. Pixel: (1, 5)

Iteration 2: $d=-2$ less than 0, so $d = -2 + 2(1) + 2 = 2, x=2, y=5$. Pixel: (2, 5)

Iteration 3: $d=2$ is greater than or equal to 0, so $d = 2 + 2(2-5) + 5 = 2 - 6 + 5 = 1, x=3, y=4$. Pixel: (3, 4)

First three pixels after (0,5): (1,5), (2,5), (3,4)